ECE 150 Fall 2023

Project 1 Staircase

Due: 11:59pm on Tuesday **September 26**, 2023

Course Material Focus

- Local Variables
- Arithmetic operators
- Conditional and comparison operators
- Loops

Estimated time to complete:

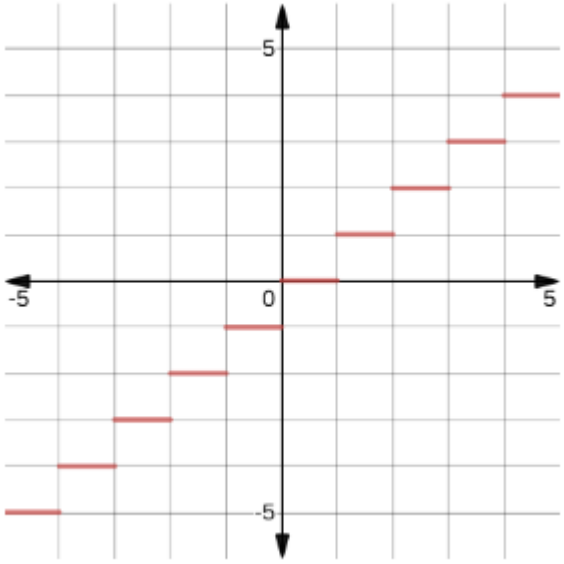
- Previous C++ experience: ~20min
- Previous Coding experience: ~30min
- No coding experience: ~40min

Introduction

A staircase function is a function defined over real values that outputs equally spaced values of equal length. While there can be several variations in its definition. For this project, we will formally define it as

$$stair(x) = \lfloor x \rfloor$$

where x ∈ ***R*** (that is, x is a real number) and ⌊x⌋ is the floor operation that accepts a real-valued number and returns the greatest integer value less than or equal to x. The plot for *stair*(x) for −5 ≤ x ≤ 5 is shown below. Be sure to note that, for example, *stair*(4.0) is the integer 4, while *stair*(3.9999999) is the integer 3.



Learning objective

The C++ library <cmath> contains a function called **floor()** which takes a decimal value and rounds down (towards negitive infinite) to the nearest integer value (e.g. 3.1 becomes 3 and -3.1 becomes -4). In this assignment you will be re-creating the functionality of the floor function using *only* the tools learned in class: simple loops, variables, and conditional operators.

Task

You will implement the *stair*(.) function starting with the given code below.

- You are not permitted to use standard math library (<math.h> or <cmath>) functions in this implementation. The only library permitted to be used is <iostream>.
- Solutions are required to use iteration (i.e. loops).
- Do not use casting (see below). Casting *forces* one data type into another.
  This can cause a loss of decimal information (i.e. truncation).
- This problem can be solved with the tools from class. It does not need string manipulation, arrays or other data types.

!! Casting: Don't do this !!

There are two types of casting:

- Explicit:
  - Using a data type in parenthesis forces the adjacent expression into that data type.
  - Example: `x = (int)3.4` will convert 3.4 into an integer, truncating the value to 3.
  - The *(int)* is an explicit cast to integer.
- Implicit:
  - Assigning a variable of one type, information of another type (causing a loss of data).

```
int x{};        // x will not store decimal information
x = 3.4;        // x stores the value 3, effectively truncating data

double y{3.4};  // y can store decimal information and stores 3.4
x = y;          // x stores the value 3, effectively truncating data

x = y % 1;      // the modulo operator % will cast y into an int
```

It is possible to use type-casting or truncation to form a solution without iteration. However, these solutions will be deemed trivial and assessed a score of zero. Be aware of possible reduction in data precision through type-casting or truncation. In general, solutions that pass Marmoset tests release16, release17, and release18 are acceptable.
If a submission fails any of these tests, you should consult with a TA to determine if the solution is acceptable or not.

Starting code

The code provided in this subsection is what you should start with, and edit. The file staircase.cpp in the Project 1 Learn folder contains this template code for you to download. You should only make modifications to the code in the portion labelled 'TASK:'.

```
#include <iostream>

int main();

int main() {
    double x{};

    std::cout << "Enter value of x: ";
    std::cin >> x;

    // This local variable should hold the result of stair( x )
    int stair{};

    // TASK: write code that calculates an integer value
    // corresponding to the value of the function stair( x ) here

    std::cout << "The value of stair(x) is: ";
    std::cout << stair;
    std::cout << std::endl;

    return 0;
}
```

Suggested procedure

You may follow this suggested procedure to complete this project.

1. Create a project using the given starting code. Compile the code in order to verify that your project is setup correctly.
2. Work out by hand what the value of the *stair*(.) is for the following argument values: `0, 0.25, 1.00, 1.99, 2.01, 2.99, 2.51, 65636.3, 65636.7, -0.25, -1.00, -1.99, -2.01, -2.99, -2.51, -65636.3, -65636.7`. Keep these next to you as you test your implementation.
3. You will now focus on modifying the starting code provided.
   a. Write the code so that it works for the argument values of 0.0 to 1.5. Test to ensure that it works correctly. *Hint: you can use an if statement to check the argument values, and appropriately return the correct value.*
   b. Write the code so that it works for values of 1.5 to 2.5. Test to ensure that it works correctly.
   c. Write the code so that it works for values of 2.5 to 3.5. Test to ensure that it works correctly.
   d. Modify the code so that it works for argument values from 0 up to and including 5.5. Test your implementation again.
4. You will now focus on generalizing your implementation.
   a. So far you used if statements with conditionals to check the argument values and respond

accordingly. Now, replace these if statements with a loop statement. Ensure that the implementation with the loop works correctly.
b. Extend the loop statement such that the code works for values up to and including 9999. Please test your implementation again.
c. Refine the loop statement so that it works for any argument value greater than 0. Test.
d. Modify the code so that it works correctly for negative values. Test.

Marmoset submission instructions

Marmoset project name: Project_1_staircase

Submission file name (case-sensitive): staircase.cpp

Once you have successfully passed all Public test cases, you will be permitted to view results of the Release tests by using a Release token. You have three Release tokens to begin with. Each Release token will renew 12 hours after it is used. In order to view Release test results, click the link labelled "Click here to release test this submission."

There may or may not be any number of Secret tests. The teaching team is instructed to not tell you whether there are any Secret tests. The results of any Secret tests will be released after the project deadline.

Plagiarism detection software

We analyze all submissions with automated plagiarism detection software. Please consult the syllabus for more information.

Feedback

Once you have completed the project we would appreciate any feedback.

Google form here: https://forms.gle/F5viGMD6rH3Zn4LY9